JMessage: A Novel Messaging System for Thomas Jefferson

Nidhish Shakya, Samuel Stankiewicz

May 29, 2026

Yilmaz - Period 3

Contents

Abstract

Recent changes in the communication landscape for Thomas Jefferson students precipitated by the exclusion of phones from school and the increasingly difficult value proposition for other external messaging apps such as Messenger (which both pose privacy and practicality concerns for TJ students) have made online communication for TJ students hard. Therefore, a third platform that can scale with and adapt to the TJ community is a practical necessity. In this project, we introduce that solution: JMessage. JMessage solves the primary pain points of traditional messaging apps to introduce a new, better solution for the TJ community. We utilize Ion authentication, which both restricts the platform exclusively to Thomas Jefferson students and creates a simple account interface, a web hosted infrastructure to allow use during the school day, and a simple design that mirrors traditional messaging platforms and increases the usability of our platform.

I. Introduction

One prevalent issue throughout the past several years at Thomas Jefferson has been a lack of a practical messaging app platform. While apps such as Messenger (Messenger. Meta Platforms, Inc.) provide a bandaid fix, they have significant deficiencies for our smaller community. For one, many students don't utilize Messenger out of privacy concerns, and participation in club activities and other extracurriculars becomes limited as a result. In addition, Messenger has been banned on FCPS devices, leaving many students without access to any form of practical communication.

This issue is non-unique to Messenger, too: texting, the phone based alternative to Messenger, poses its own issues. Namely, messages are only usable on a phone or Apple laptop. Phones are banned during the school day, and this therefore segments the TJ community into communicating only if the end user owns an Apple device. For a widescale solution designed to be adaptable to the community as a whole, traditional texting falls short.

Therefore, our project focuses on identifying a third solution. Our central question: how can we create a messaging app with a similar ease of use to traditional messaging platforms while retaining the privacy benefits for the TJ community? An implementation must be both familiar to what students are already accustomed to with Messenger or iMessage while adapting it to be more practical for students and school. Our solution: JMessage.

II. Background

Building a messaging app requires the usage of a frontend-backend system where a server, or a backend, receives changes & actions that are made by a user from the ui, frontend, to store information into a database. From there, when the user “asks” to pull information from the server, the server can query the database for any given information; this would include chats, messages, and even users. Messaging apps, such as messenger or IMessage, are built off this similar model, though often with an additional level of security & encryption to fully employ End-to-End encryption.

III. Applications

As a messaging app, the primary application would be to message other students. Outside of that, our app could work as an announcement board for clubs. Since group chats are implemented, you could make a group chat with all club members and use that chat as a form of news; we believe would be more effective than just using Ion announcements as people are much more likely to keep track of a chat when compared to Ion webmail or announcements.

IV. Methods

We utilize several frameworks to build this application: namely, Node.js and Express for server code, and EJS as a frontend development framework. EJS is beneficial as compared to standard HTML because of the improved JavaScript injection functionality, which more easily allows us to load messages and user information from the server. All backend (server) code is written in JavaScript. Additionally, we use MongoDB as our database and Mongoose as a backend scripting interface that allows for communication between the hosted and database servers.

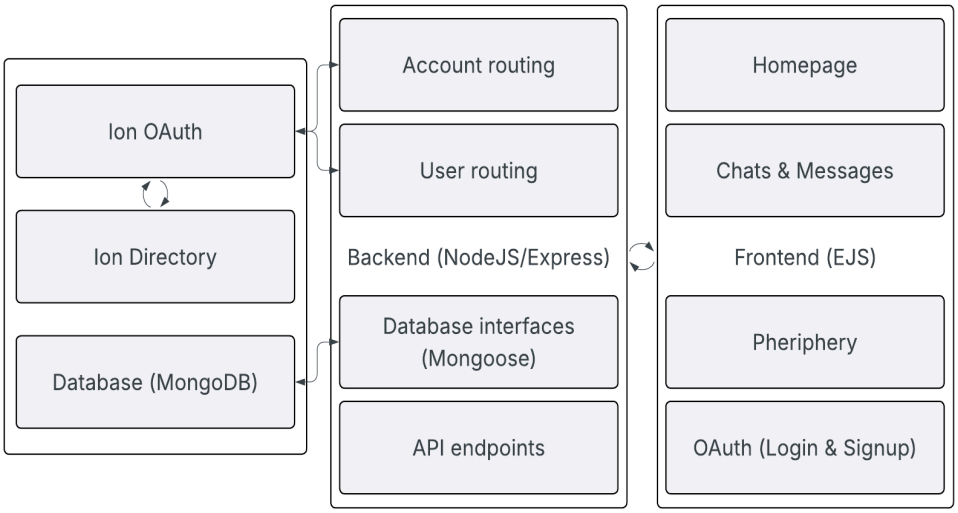
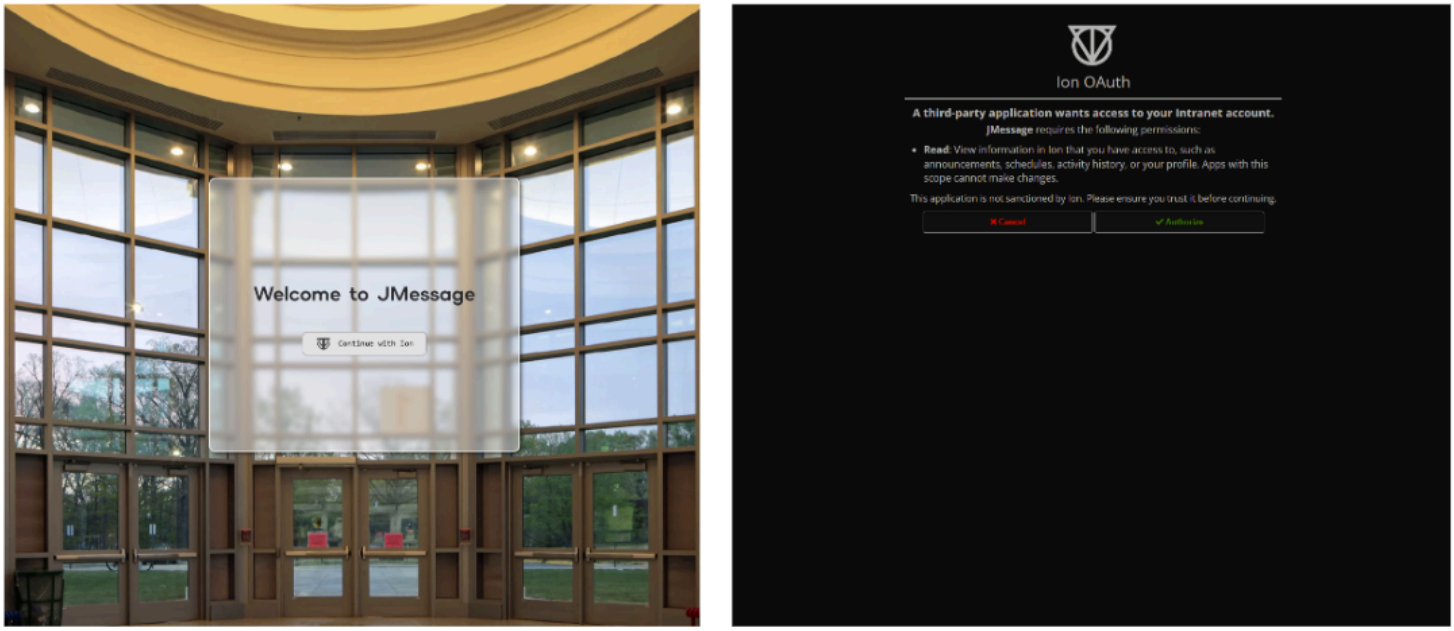


Fig 1. Technical Interaction Flow Diagram, displaying the frontend-backend system. The frontend accounts for user activity, such as creating chats or sending a message, whereas the backend receives that activity to input into the database (MongoDB). As represented through the two-headed arrows, when a user performs an action that requires stored data, such as opening a chat with another user, the backend queries and pulls information from our database to send forwards.

One of the critical differentiators between our project and a traditional, large scale messenger is the use of Ion authentication, which both restricts use of our platform to the TJ community and allows all TJ students simple access to the platform. As seen on Fig. 1, when a user logs in (Account Routing) the backend requests authorization through IonOAuth, which verifies a user via the Ion Directory. Note that Ion strictly controls the extent of the data we're given, so private login information is not shared.

V. Results

Our resulting app is currently hosted on [trulycalc.com](), which is Sam's personal domain. With the use of render, the app is able to be launched by anyone, at any time, given that the host itself is functioning. As stated previously, user login is controlled via IonOAuth/Ion Director directly, as such there is no need for the implementation of a fully fleshed out login page.



a) Unauthorized home page

b) IonOAuth user verification

Fig. 2. JMessage login interface. An unauthorized user is initially asked to continue with Ion, only then is a user redirected to OAuth to request verification. Again, Ion strictly controls and verifies the user, as such private login information, like a password, is unavailable to JMessage.

Rather the server can only request auxiliary information such as a name or grade level from the Ion Directory.

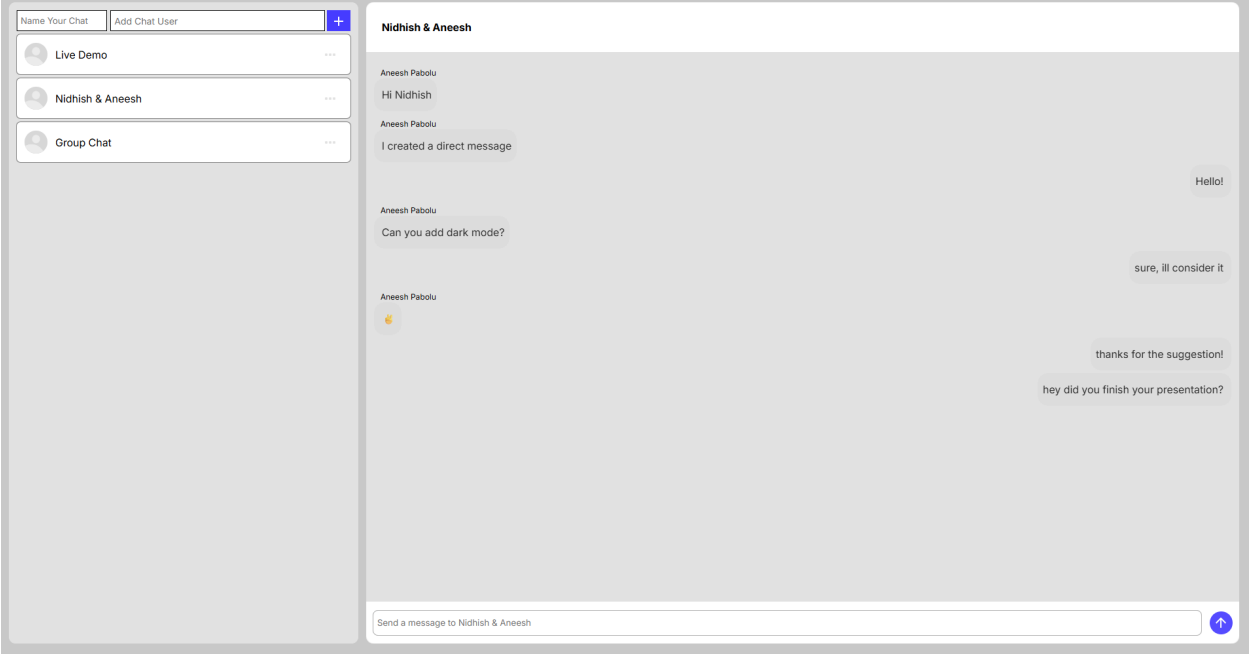


Fig 3. JMessage chats home page. Once a user is authenticated through IonOAuth, they will be redirected to a chats page. From here, the user is free to send a message in any selected chat, or also create a new chat with another user. A feature to note is the full implementation of websockets; allowing for real-time updates on messages with the need for manual refreshing.

VI. Limitations

A major limitation for JMessage comes from the inherently weak server host. Since JMessage is currently run on a personal server, with the use of render, the strength of that server may not be strong enough to handle a large number of active users. While the server load can be reduced through using web sockets, for example, instead of constant polling, ultimately having multiple users query the database multiple times per second, will eventually create a strain too heavy.

In addition to logistical issues, JMessage is also not fully secured. While important Ion data is stored on IonOAuth/Ion Directory, the lack of proper encryption and sanitization on JMessage, means that important information a user may send to another user may not be fully secured. That being said, the fix for both logistical issues and security issues are rather straightforward; there's only a n implication of time. As such, given future efforts to secure JMessage, will lead success.

Nonetheless, another major limitation could be the reliance on Ion. As Ion serves as JMessage's infrastructure for user authentication, if Ion were to either crash or go down, then JMessage will follow suit. There is no real solution to this limitation, and serves as the main Achilles' Heel.

VII. Conclusion

Through the implementation of IonOAuth, JMessage is a functional alternative to other messaging platforms often used by TJ students.Using messenger as a boilerplate, or as a model, we created an application that students will already feel comfortable with, with the added benefit of allowing for new features that prioritize TJ students. IonOAuth allows for speedy account creation while also verifying users, while the implementation of web sockets allows for efficient messaging. Concerns about message security still play a role in our minds as we look for new ways to improve JMessage, but it is with doubt that JMessage is at least functional as a messaging app.

VIII. Future Work

As mentioned previously, the biggest step to improve JMessage comes in the form of added security. Currently, the encryption for messages is nonexistent, and message sanitization is at a bare minimum. If someone were to, maliciously, seek some way to hack into JMessage, either to look into another user's personal information or chats, there'd be very few measures to prevent them. If encryption methods, as described in [1] for instance, were to be implemented, JMessage's security would be exponentially improved.

In addition to further security measures, another major addition would be to JMessage's searching functionality. As we created JMessage, we focused more so on functionality rather than usability. Currently, the searching can feel convoluted, especially to users who aren't given any direct guidance by someone else who has used JMessage. More so, having the capability to make group chats is a major searching-adjacent feature that should be prioritized. We've already tested that group chats work, but the issue is that users don't actually have a way to make them. Right now, we must manually add users to their respective chats through our database to make group chats. While security is the largest issue, fixes to searching should also be duly noted as quality-of-life additions.

IX. References

[1] N. Kiran, "Securing Chat applications: Strategies for end-to-end encryption and cloud data protection," *World Journal of Advanced Engineering Technology and Sciences*, vol. 13, no. 2, pp. 528–540, Dec. 2024, doi: https://doi.org/10.30574/wjaets.2024.13.2.0634.

X. Materials

Nidhish's Github Repo: https://github.com/Dishes00/JMessageFinalPackage
Sam's Github Repo: https://github.com/SamlKeller/JMessage